

claude-windows / 902e27a6-e026-4b55-8c26-f2c47329cc44

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\902e27a6-e026-4b55-8c26-f2c47329cc44\subagents\workflows\wf\_c287c071-8e9\agent-a3561f765bd1892ef.jsonl`
- SHA-256: `e199c09e3d4d9ccc40cab23c6fbdc9934f1f7e6f34de039bb5d4f78e6e4a133e`
- Source modified: `2026-07-07T14:51:25+00:00`
- Imported at: `2026-07-08T16:00:01+00:00`
- project: `wf\_c287c071-8e9`
- session_id: `902e27a6-e026-4b55-8c26-f2c47329cc44`

Transcript

1. user (2026-07-07T14:48:49.998Z)

You are reviewing an uncommitted change in a git worktree at:
C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval
Branch: fix/worker-skip-revalidation-on-dispatch (based on origin/master).

THE BUG BEING FIXED: On the cloud-serving path, the control-plane (backend/orchestration.py) runs validators and ONLY dispatches to the Cloud Run worker on PASS. The worker (backend/worker/__main__.py cmd_infer) then REDUNDANTLY re-ran validation with its OWN config (different min_blur_threshold), rejecting a clip the control-plane already passed. Live failure 2026-07-07: passed at min_blur_threshold=8.0 on the control-plane, worker rejected at its default 20.0 ? GPU dispatch succeeded but the CUJ failed with worker rc=2.

THE FIX (skip-on-dispatch): a new ComputeJobSpec.skip_validation bool (default False) ? backend/compute/cloud_run.py emits a "--skip-validation" worker CLI flag when set ? backend/worker/__main__.py cmd_infer honors it (skips validator_registry.run_all_with_context, sets val_results=[], val_context={}) ? backend/orchestration.py::_maybe_dispatch_remote sets skip_validation=True on every dispatch. The direct "worker infer" CLI (no control-plane) still validates by default. New tests in tests/test_worker.py and tests/test_api_cloud_dispatch.py.

Inspect the ACTUAL code ? do NOT trust this summary. Run: cd "C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval" && git --no-pager diff and read the surrounding functions (not just the diff hunks). The full test suite already passes (1779 passed), ruff + mypy are clean, coverage 85.42%. So do NOT report style nits or things the gate already covers. Hunt for REAL correctness/rollout/consumer defects the tests would miss. Only report a finding you can tie to a concrete failure scenario in THIS repo's code.

A reviewer (lens: tests) raised this finding about the change. ADVERSARIALLY verify it against the REAL code in C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval (read the files; run git grep / git diff). Default to REFUTED unless you can demonstrate the concrete failure in THIS repo's code. If real, give the minimal fix.

FINDING:
severity=major category=test-coverage
file=tests/test_compute_backend.py line=201
summary=No test asserts that a default spec (skip_validation=False) OMTS --skip-validation from the worker argv, leaving the 'direct CLI still validates on the cloud path' safety invariant unguarded at the argv builder.
detail=cloud_run.py::run_infer (backend/compute/cloud_run.py:199-202) appends '--skip-validation' gated on `if spec.skip\_validation:`. Every test that exercises argv building either uses a default spec without asserting the flag's ABSENCE

(test_dispatch_argv_forces_inline_and_carries_overrides, line 201, builds a spec with skip_validation at its default False but never checks `--skip-validation` not in argv`) or goes through the control-plane which always sets True (all test\_cloud\_dispatch\_\* in test\_api\_cloud\_dispatch.py). So the invariant the change explicitly promises to preserve -- 'Default False keeps the direct `worker infer` CLI validating exactly as before' -- has no regression guard at the builder that constructs the flag. Add an assertion to test_dispatch_argv_forces_inline_and_carries_overrides (or a dedicated test) that a default ComputeJobSpec yields argv WITHOUT '--skip-validation', and a paired one that skip_validation=True yields it.

failure_scenario=A future refactor of run_infer's argv construction drops or inverts the `if spec.skip\_validation:` guard (e.g. appends the flag unconditionally, or a bad merge). The full suite still passes green: the only argv-builder test uses a default spec and never asserts the flag is absent, and every control-plane dispatch test legitimately expects True. The remote worker on the direct-CLI cloud dispatch path (_dispatch_remote, skip_validation=False) then silently skips validation -- reintroducing exactly the class of 'validation was bypassed when it shouldn't be' defect this change is guarding, undetected.

2. assistant (2026-07-07T14:48:54.598Z)

I'll inspect the actual code to verify this finding about test coverage.

3. user (2026-07-07T14:48:56.767Z)

```
diff --git a/backend/compute/base.py b/backend/compute/base.py
index 279606e..31af3fe 100644
--- a/backend/compute/base.py
+++ b/backend/compute/base.py
@@ -28,6 +28,13 @@ class ComputeJobSpec:
     segmenter: Optional[str] = None
     # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
     frozen/hashable.
     config_overrides: tuple[str, ...] = ()
+    # Skip the worker's redundant re-validation. The DISPATCHER (the cloud-serving control-
+    plane) is
+    # the authoritative validation gate: it runs the SAME validators with ITS resolved config
+    and only
+    # dispatches on PASS, so a worker re-run is (a) redundant, (b) a second full-video decode,
+    and
+    # (c) outright WRONG when the worker's config resolves different thresholds ? it can REJECT
+    a clip
+    # the gate already passed. Default False keeps the direct ``worker infer`` CLI (no control-
+    plane)
+    # validating exactly as before.
+    skip_validation: bool = False
```

```
@dataclass(frozen=True)
diff --git a/backend/compute/cloud_run.py b/backend/compute/cloud_run.py
index 841785d..7247ace 100644
--- a/backend/compute/cloud_run.py
+++ b/backend/compute/cloud_run.py
@@ -196,6 +196,10 @@ class CloudRunCompute(ComputeBackend):
     ]
     if spec.segmenter:
         argv += ["--segmenter", spec.segmenter]
+    if spec.skip_validation:
+        # The control-plane already validated + only dispatches on PASS ? tell the worker
+        not to
+        # re-decode and re-validate (redundant, and its config may resolve different
+        thresholds).
+        argv.append("--skip-validation")
     for kv in (*spec.config_overrides, *self._container_overrides):
         argv += ["--set", kv]
```

```
diff --git a/backend/orchestration.py b/backend/orchestration.py
```

```

index 5e9f510..79b4fed 100644
--- a/backend/orchestration.py
+++ b/backend/orchestration.py
@@ -568,11 +568,18 @@ def _maybe_dispatch_remote(
    prefetch = getattr(wasb_cfg, "prefetch_batches", None)
    if prefetch:
        overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
+   # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
authoritative
+   # gate ? it already ran the validators with its resolved config just above (only a PASS
reaches
+   # here; a failure returned "failed" before dispatch). A worker re-run would decode the
video a
+   # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
+   # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
+   # then worker-rejected at its default 20.0 after a successful GPU dispatch).
    spec = ComputeJobSpec(
        video_uri=req.video_path,
        out_uri=out_root,
        segmenter=seg_name,
        config_overrides=tuple(overrides),
+       skip_validation=True,
    )
    try:
        # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
index 88c50f6..9b19215 100644
--- a/backend/worker/__main__.py
+++ b/backend/worker/__main__.py
@@ -186,6 +186,7 @@ def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
    out_uri=args.out_uri,
    segmenter=args.segmenter,
    config_overrides=tuple(args.set or ()),
+    skip_validation=bool(getattr(args, "skip_validation", False)),
    )
    try:
        result = compute.run_infer(spec)
@@ -259,10 +260,25 @@ def cmd_infer(args: argparse.Namespace) -> int:
    # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be byte-
identical).
    video_id = compute_video_id(local_video)

-   # 3. VALIDATE (same registry as the main CLI).
-   val_results, val_context = validator_registry.run_all_with_context(
-       local_video, config
-   )
+   # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
+   # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate: it runs
the
+   # SAME validators with ITS resolved config and only dispatches on PASS. Re-validating here
is
+   # redundant, decodes the whole video a SECOND time, and is outright WRONG when the worker's
config
+   # resolves different thresholds than the control-plane's ? observed 2026-07-07: a 5312x2988
clip
+   # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0, then the
worker
+   # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the fact.
+   # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote) skips
it; the
+   # direct ``worker infer`` CLI has no control-plane, so it still validates by default.
+   if getattr(args, "skip_validation", False):
+       print(

```

```

+         "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
+         "gate and already validated this input."
+     )
+     val_results, val_context = [], {}
+     else:
+         val_results, val_context = validator_registry.run_all_with_context(
+             local_video, config
+         )
+         failures = [r for r in val_results if not r.passed]
+         db = Database(os.path.join(scratch, config.output.db_name))
+         db.add_video(
@@ -724,6 +740,14 @@ def build_parser() -> argparse.ArgumentParser:
+             metavar="k=v",
+             help="config override, e.g. --set indexing.skip_ai_handoff=true",
+         )
+     pi.add_argument(
+         "--skip-validation",
+         action="store_true",
+         help="skip the worker's re-validation: the dispatcher (control-plane) already ran the "
+         "validators with its resolved config and only dispatches on PASS, so re-validating here
is "
+         "redundant, decodes the video again, and can contradict the gate. Default-OFF: the
direct "
+         "`worker infer` CLI still validates.",
+     )
+     pi.set_defaults(func=cmd_infer)

    pt = sub.add_parser(
diff --git a/tests/test_api_cloud_dispatch.py b/tests/test_api_cloud_dispatch.py
index 8382234..19c37d9 100644
--- a/tests/test_api_cloud_dispatch.py
+++ b/tests/test_api_cloud_dispatch.py
@@ -236,6 +236,22 @@ def test_cloud_dispatch_omits_unset_wasb_tuning(cloud_env):
    assert "indexing.wasb.prefetch_batches" not in joined

+def test_cloud_dispatch_skips_worker_revalidation(cloud_env):
+    """The control-plane is the AUTHORITATIVE validation gate: it validated + only dispatches
on PASS
+    (registry.run_all_with_context is stubbed to pass in cloud_env). Re-validating on the
worker is
+    redundant, decodes the video a second time, and ? because the control-plane's validation
config
+    isn't forwarded ? can REJECT a clip the gate already passed (observed 2026-07-07: passed at
+    validation.min_blur_threshold=8.0, then worker-rejected at its default 20.0). So the
dispatch MUST
+    carry --skip-validation, telling the worker to trust the gate. Regression guard for that
wiring."""
+    _db, _bucket, monkeypatch = cloud_env
+    fake = _FakeCloudClient()
+    _inject_fake(monkeypatch, fake)
+    m._execute_processing(
+        m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
+    )
+    assert "--skip-validation" in fake.calls[0]
+
+def test_cloud_failure_marks_failed_no_rows(cloud_env):
+    db, _bucket, monkeypatch = cloud_env
+    # pre-seed a prior good row ? must be preserved on a cloud failure (destroy-AFTER-confirm).
diff --git a/tests/test_worker.py b/tests/test_worker.py
index 8fb5e16..72b2514 100644
--- a/tests/test_worker.py
+++ b/tests/test_worker.py

```

```

@@ -97,6 +97,14 @@ def test_parser_infer_accepts_set_and_uris():
    assert args.set == ["indexing.skip_ai_handoff=true", "sport=tennis"]

+def test_parser_infer_skip_validation_flag():
+    base = ["infer", "--video-uri", "s3://b/v.mp4", "--out-uri", "s3://b"]
+    # Default-OFF: the direct `worker infer` CLI still validates.
+    assert wmain.build_parser().parse_args(base).skip_validation is False
+    # The dispatcher passes it to disable the redundant worker re-validation.
+    assert wmain.build_parser().parse_args(base + ["--skip-validation"]).skip_validation is
True
+
+
# ----- cmd_infer (local, mocked)

@@ -407,6 +415,43 @@ def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
    assert marker["returncode"] == 2 and marker["status"] == "failed"

+def test_cmd_infer_skip_validation_bypasses_validators(tmp_path, monkeypatch):
+    """--skip-validation (set by the dispatcher, the authoritative gate) must NOT run the
worker's
+    validators ? no second decode, and no chance of a threshold mismatch REJECTING a clip the
control-plane already passed (the live 2026-07-07 CUJ failure). Prove it by making
+    run_all_with_context RAISE: a skipped run never touches it, so the job completes rc 0 with
a
+    status="success" report. Contrast test_cmd_infer_validation_failure_returns_2 (no flag ? rc
2)."""
+    monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidSkip")
+
+    def _boom(*a, **k): # the validators must never be reached on the skip path
+        raise AssertionError("run_all_with_context must not run under --skip-validation")
+
+    monkeypatch.setattr(wmain.validator_registry, "run_all_with_context", _boom)
+    monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FakeSeg)
+    monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
+    video = tmp_path / "in.mp4"
+    video.write_bytes(b"FAKE")
+    out = tmp_path / "bucket"
+
+    rc = wmain.main(
+        [
+            "infer",
+            "--video-uri",
+            str(video),
+            "--out-uri",
+            str(out),
+            "--scratch",
+            str(tmp_path / "s"),
+            "--segmenter",
+            "wasb_hybrid",
+            "--skip-validation",
+        ]
+    )
+    assert rc == 0 # a clip that WOULD fail re-validation still succeeds ? the gate is trusted
+    report = json.loads((out / "outputs" / "vidSkip" / "report.json").read_text())
+    assert report["status"] == "success" and report["segment_count"] == 2
+
+
# --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle ---

```

4. assistant (2026-07-07T14:49:01.869Z)

Let me look at the test file referenced in the finding and the argv-building test.

5. user (2026-07-07T14:49:04.613Z)

```
tests/test_api_cloud_dispatch.py:245:    carry --skip-validation, telling the worker to trust
the gate. Regression guard for that wiring."""
tests/test_api_cloud_dispatch.py:252:    assert "--skip-validation" in fake.calls[0]
tests/test_worker.py:100: def test_parser_infer_skip_validation_flag():
tests/test_worker.py:103:    assert wmain.build_parser().parse_args(base).skip_validation is
False
tests/test_worker.py:105:    assert wmain.build_parser().parse_args(base + ["--skip-
validation"]).skip_validation is True
tests/test_worker.py:418: def test_cmd_infer_skip_validation_bypasses_validators(tmp_path,
monkeypatch):
tests/test_worker.py:419:    """--skip-validation (set by the dispatcher, the authoritative
gate) must NOT run the worker's
tests/test_worker.py:427:        raise AssertionError("run_all_with_context must not run under
--skip-validation")
tests/test_worker.py:447:        "--skip-validation",
```

6. user (2026-07-07T14:49:04.750Z)

```
150         client.run_job(
151             "rally-worker", ["infer"], region="asia-south1", project=missing
152         )
153
154
155     def test_real_client_cancel_rejects_unresolvable_execution_name():
156         """cancel_execution must fail loudly on a non-resolvable execution name ? run_job
falls back
157         to the literal "execution" when the operation metadata carries no name, and
cancelling that
158         would be a bogus API call. The guard runs BEFORE the google-cloud-run import
(mirrors the
159         project guard), so it is exercisable in CI (the SDK is intentionally not a test
dependency)."""
160         from backend.compute.cloud_run import GoogleCloudRunJobClient
161
162         client = GoogleCloudRunJobClient()
163         for bogus in ("", "execution", "exec-123"):
164             with pytest.raises(RuntimeError, match="fully-qualified"):
165                 client.cancel_execution(bogus)
166
167
168     # ----- CloudRunCompute dispatch
+ poll
169
170
171     def test_cloud_run_dispatch_polls_and_returns_result(tmp_path):
172         client = _FakeRunClient(video_id="abc123", segment_count=3)
173         backend = CloudRunCompute(
174             run_client=client,
175             job_name="rally-worker",
176             region="asia-south1",
177             project="proj",
178             storage_config={},
179             policy=_FAST,
180             token="tok",
181         )
182         out = str(tmp_path / "bucket")
183         result = backend.run_infer(
184             ComputeJobSpec(
185                 video_uri="gs://b/videos/clip.mp4", out_uri=out, segmenter="wasb_hybrid"
186             )
187         )
188
189         assert result.returncode == 0
```

```

190     assert result.video_id == "abc123"
191     assert result.outputs_uri == f"{out}/outputs/abc123/"
192     assert result.report.get("segment_count") == 3
193     # exactly one Cloud Run Job execution was triggered, in the right region/project
194     assert len(client.calls) == 1
195     assert (
196         client.calls[0]["job"] == "rally-worker"
197         and client.calls[0]["region"] == "asia-south1"
198     )
199
200
201     def test_dispatch_argv_forces_inline_and_carries_overrides(tmp_path):
202         """The dispatched container must run INLINE (never re-dispatch) ?
compute_target=local_gpu +
203         native detector appended AFTER the user overrides (so they win); --done-uri always
present."""
204         client = _FakeRunClient()
205         backend = CloudRunCompute(
206             run_client=client, job_name="j", region="r", policy=_FAST, token="tok"
207         )
208         backend.run_infer(
209             ComputeJobSpec(
210                 video_uri="gs://b/v.mp4",
211                 out_uri=str(tmp_path / "bk"),
212                 segmenter="wasb_hybrid",
213                 config_overrides={"sport=tennis"},
214             )
215         )
216
217         argv = client.calls[0]["argv"]
218         assert argv[0] == "infer"
219         assert "--done-uri" in argv and "--video-uri" in argv and "--out-uri" in argv
220         assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
221         # the container-inline overrides come last (win) and the user override is carried
through
222         joined = " ".join(argv)
223         assert "deployment.compute_target=local_gpu" in joined
224         assert "indexing.detector_impl=native" in joined
225         assert "sport=tennis" in joined
226         # user override precedes the inline-forcing overrides in --set order
227         assert joined.index("sport=tennis") < joined.index(
228             "deployment.compute_target=local_gpu"
229         )
230
231
232     def test_unique_token_per_call_when_not_injected(tmp_path):
233         """No injected token ? a fresh uuid per dispatch, so concurrent jobs don't collide
on the marker."""
234         client = _FakeRunClient()
235         backend = CloudRunCompute(run_client=client, job_name="j", region="r", policy=_FAST)
236         backend.run_infer(
237             ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk"))
238         )
239         backend.run_infer(
240             ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk"))
241         )
242         done1 = client.calls[0]["argv"][client.calls[0]["argv"].index("--done-uri") + 1]
243         done2 = client.calls[1]["argv"][client.calls[1]["argv"].index("--done-uri") + 1]
244         assert done1 != done2
245
246
247     def test_read_json_passes_dst_for_cloud_backends(monkeypatch):
248         """BLOCKER guard: GCS/S3 fetch_to_local REQUIRES a dst (a None dst raises) ?
_read_json must
249         pass one, else every real cloud marker/report read silently returns {} (local hid

```

```

this via its
250         identity passthrough). Mimic a cloud backend that rejects dst=None.""
251         from backend.compute import cloud_run as cr
252
253         captured = {}
254
255         def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
256             captured["dst"] = dst
257             if dst is None:
258                 raise ValueError(
259                     "cloud backend requires a dst path"
260                 ) # mirror GCSStorage/S3Storage
261             with open(dst, "w", encoding="utf-8") as f:
262                 f.write('{"video_id": "v", "returncode": 0}\'')
263             return dst
264
265         monkeypatch.setattr(cr, "fetch", fake_fetch)
266         backend = cr.CloudRunCompute(run_client=_FakeRunClient(), job_name="j", region="r")
267         data = backend._read_json("gs://b/x.json")
268         assert (
269             captured["dst"] is not None

```

7. assistant (2026-07-07T14:49:34.122Z)

Let me examine the argv-building tests more thoroughly and check whether any test would catch an unconditional-append regression.

8. user (2026-07-07T14:49:37.348Z)

```

tests/test_compute_backend.py:183:         result = backend.run_infer(
tests/test_compute_backend.py:196:             client.calls[0]["job"] == "rally-worker"
tests/test_compute_backend.py:197:             and client.calls[0]["region"] == "asia-south1"
tests/test_compute_backend.py:208:         backend.run_infer(
tests/test_compute_backend.py:217:             argv = client.calls[0]["argv"]
tests/test_compute_backend.py:236:         backend.run_infer(
tests/test_compute_backend.py:239:         backend.run_infer(
tests/test_compute_backend.py:242:         done1 =
client.calls[0]["argv"][client.calls[0]["argv"].index("--done-uri") + 1]
tests/test_compute_backend.py:307:         result = backend.run_infer(
tests/test_compute_backend.py:334:             backend.run_infer(
tests/test_compute_backend.py:356:             backend.run_infer(
tests/test_compute_backend.py:393:         result = backend.run_infer(
tests/test_compute_backend.py:413:             def run_infer(self, spec):
tests/test_compute_backend.py:754: def
test_run_infer_forwards_on_wait_into_the_bucket_poll(tmp_path, monkeypatch):
tests/test_compute_backend.py:755:     """The #482 heartbeat plumbing: the on_wait callback
handed to run_infer must be the
tests/test_compute_backend.py:779:         result = backend.run_infer(

```

9. user (2026-07-07T14:49:37.636Z)

```

1         """API cloud-serving dispatch (COMPUTE_DECOUPLED_SERVING M6 ? the API path).
2
3         Proves the /api/process job path can offload DETECT to a Cloud Run GPU Job and ingest
the rally
4         intervals back into the API DB, so /api/query + /api/compile (local stitch) are
unchanged ? all
5         GPU-free + cloud-free via a FAKE CloudRunJobClient that writes report.json +
intervals.csv + the
6         done-marker into a local "bucket". Cardinal invariant: DEFAULT-OFF ? only
compute_target=cloud_run
7         dispatches; every other target runs the in-process segmenter byte-identically.
8         """
9

```

```

10     import csv
11     import json
12     import os
13
14     import pytest
15
16     pytest.importorskip("httpx")
17     from fastapi.testclient import TestClient
18
19     import backend.compute as compute_pkg
20     import backend.main as m
21     from backend.compute.cloud_run import CloudRunJobClient
22     from backend.config.models import AppConfig
23     from backend.infrastructure.database import Database
24     from backend.infrastructure.execution_policy import ExecutionPolicy
25     from backend.pipeline.validators.base import ValidationResult
26
27     _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
28
29
30     def _pass():
31         return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]
32
33
34     class _InlineExecutor:
35         def submit(self, fn, *a, **k):
36             fn(*a, **k)
37
38
39     class _FakeCloudClient(CloudRunJobClient):
40         """Simulates the Cloud Run worker: on dispatch, write report.json + intervals.csv +
the done-marker
41         into the local bucket exactly where the real worker would, so the bucket-poll +
ingest resolve."""
42
43         def __init__(self, *, video_id="vidCloud", returncode=0, rows=None):
44             self.calls: list[list[str]] = []
45             self.video_id, self.returncode = video_id, returncode
46             self.rows = (
47                 rows
48                 if rows is not None
49                 else [(2.74, 6.91, 5, "winner"), (8.74, 11.38, 3, "unforced_error")]
50             )
51
52         def run_job(self, job_name, argv, *, region, project=None):
53             self.calls.append(list(argv))
54             out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
55             done_uri = argv[argv.index("--done-uri") + 1]
56             if self.returncode == 0:
57                 outputs = os.path.join(out_uri, "outputs", self.video_id)
58                 os.makedirs(outputs, exist_ok=True)
59                 with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
60                     json.dump(
61                         {
62                             "video_id": self.video_id,
63                             "segment_count": len(self.rows),
64                             "status": "success",
65                         },
66                         f,
67                     )
68                 with open(
69                     os.path.join(outputs, "intervals.csv"),
70                     "w",
71                     newline="",
72                     encoding="utf-8",

```

```

73         ) as f:
74             w = csv.writer(f)
75             w.writerow(["start", "end", "shot_count", "ending_reason"])
76             for s, e, sc, er in self.rows:
77                 w.writerow([f"{s:.3f}", f"{e:.3f}", sc, er])
78         os.makedirs(os.path.dirname(done_uri), exist_ok=True)
79         with open(done_uri, "w", encoding="utf-8") as f:
80             json.dump(
81                 {
82                     "video_id": self.video_id,
83                     "returncode": self.returncode,
84                     "segment_count": len(self.rows),
85                     "status": "success",
86                 },
87                 f,
88             )
89         return "exec-1"
90
91     def cancel_execution(self, execution):
92         raise AssertionError(
93             "cancel must never fire on the happy path"
94         ) # pragma: no cover
95
96
97     @pytest.fixture
98     def cloud_env(tmp_path, monkeypatch):
99         """API wired for cloud_run with a LOCAL bucket (the local storage backend makes the
100         bucket read an
101         identity file read ? no GCS). A gs:// input is 'fetched' to a local fake file for
102         hash+validate."""

```

10. assistant (2026-07-07T14:50:15.518Z)

Let me verify the failure scenario concretely: would appending `--skip-validation` unconditionally break ANY test in the suite? Let me search across the whole test suite for exact-argv assertions or a real worker consuming the built argv.

11. user (2026-07-07T14:50:19.449Z)

```

---- run_all_with_context call sites in worker ----
backend/compute/base.py:37:     skip_validation: bool = False
backend/compute/cloud_run.py:199:         if spec.skip_validation:
backend/main.py:1513:         val_results, val_context = registry.run_all_with_context(
backend/orchestration.py:582:         skip_validation=True,
backend/orchestration.py:754:         val_results, val_context =
registry.run_all_with_context(local_video, config)
backend/pipeline/validators/base.py:50:     def run_all_with_context(self, video_path: str,
config: Any):
backend/pipeline/validators/base.py:78:         results, _ =
self.run_all_with_context(video_path, config)
backend/worker/__main__.py:189:         skip_validation=bool(getattr(args, "skip_validation",
False)),
backend/worker/__main__.py:272:         if getattr(args, "skip_validation", False):
backend/worker/__main__.py:279:         val_results, val_context =
validator_registry.run_all_with_context(

```

12. user (2026-07-07T14:50:29.262Z)

```

tests/test_worker.py:100: def test_parser_infer_skip_validation_flag():
tests/test_worker.py:103:     assert wmain.build_parser().parse_args(base).skip_validation is
False
tests/test_worker.py:105:     assert wmain.build_parser().parse_args(base + ["--skip-
validation"]).skip_validation is True
tests/test_worker.py:418: def test_cmd_infer_skip_validation_bypasses_validators(tmp_path,

```

monkeypatch):

13. user (2026-07-07T14:50:38.038Z)

```
tests/test_worker.py:1: """Worker dispatcher: pull (storage) ? run pipeline ? push (storage).
tests/test_worker.py:104: # The dispatcher passes it to disable the redundant worker re-
validation.
tests/test_worker.py:295: done = tmp_path / "bucket" / "_dispatch" / "tok.done"
tests/test_worker.py:318: # ...and the completion marker the cloud dispatcher bucket-polls
carries the same verdict.
tests/test_worker.py:354: done = tmp_path / "bucket" / "_dispatch" / "tok.done"
tests/test_worker.py:393: done = tmp_path / "o" / "_dispatch" / "tok.done"
tests/test_worker.py:419: """--skip-validation (set by the dispatcher, the authoritative
gate) must NOT run the worker's
```

14. assistant (2026-07-07T14:51:11.233Z)

I have thoroughly verified the finding against the actual code. Let me confirm my analysis of the key facts.

Verified facts:

1. ``backend/compute/cloud_run.py:199`` ? the flag is appended gated on ``if spec.skip_validation:`` (3-line block).
2. ``test_dispatch_argv_forces_inline_and_carries_overrides`` (tests/test_compute_backend.py:201) builds a spec with ``skip_validation`` at its default ``False`` and asserts only via ``in`/index` checks (``--done-uri``, ``--segmenter``, overrides) ? it never asserts ``"--skip-validation"` not in `argv``.
3. Grep across all tests: the only assertions of ``--skip-validation`` presence are (a) ``test_cloud_dispatch_skips_worker_revalidation`` (goes through the control-plane, which hardcodes ``skip_validation=True`` in ``orchestration._maybe_dispatch_remote:582``), (b) ``test_parser_infer_skip_validation_flag`` (parser-level, not argv builder), (c) ``test_cmd_infer_skip_validation_bypasses_validators`` (worker behavior). No test asserts the flag's ABSENCE at the argv builder for a default spec.
4. No test in the suite does an exact-argv equality/length/count assertion, and no test has a real worker parse the built argv (the worker is faked in both ``test_compute_backend.py`` and ``test_api_cloud_dispatch.py``). So an unconditional-append mutation of the guard would not break any ``in`/index-based` check.

One nuance that narrows the finding: the "inverts the guard" variant of the failure scenario IS already caught ? ``test_cloud_dispatch_skips_worker_revalidation`` expects the flag present when the control-plane sets ``True``, so inverting ``if spec.skip_validation:`` fails that test. Only the "drop the guard / append unconditionally" mutation is genuinely uncaught.

15. user (2026-07-07T14:51:25.634Z)

Structured output provided successfully